

从 OpenClaw 到 AstronClaw

Agent 如何变得更好用、更落地

为什么很多人觉得 OpenClaw 不好用？

不是产品的问题，是使用姿势没对。

1. 模型没选对

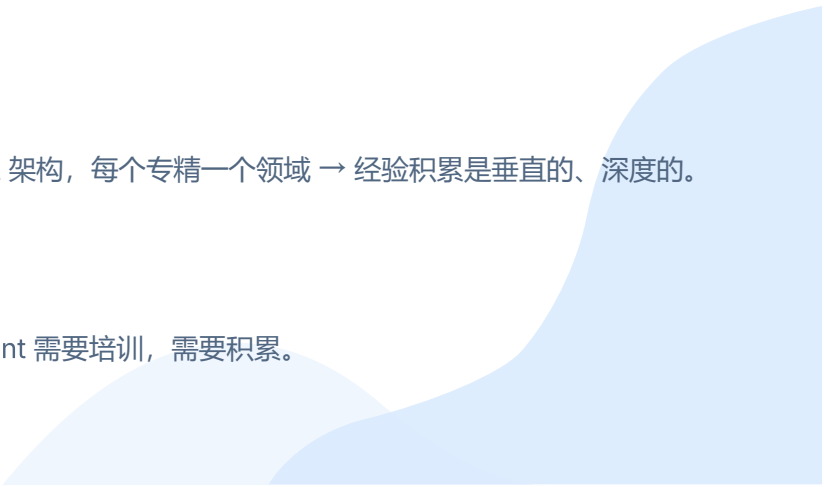
OpenClaw 本身不生产智能，它是让 AI 模型发挥得更好的**框架**。给实习生写操作手册 vs 给高级工程师写操作手册，效果天差地别。

2. 把 Agent 当成通才

一个 agent 什么都干 → 经验积累是水平的、稀薄的。多 Agent 架构，每个专精一个领域 → 经验积累是垂直的、深度的。

3. 没有"培训"你的 Agent

开箱就用，等于新员工第一天就指望他跟老员工一样好使。Agent 需要培训，需要积累。

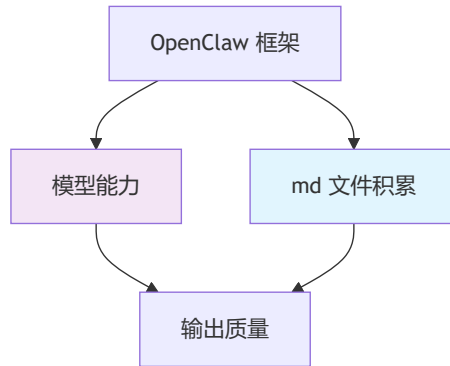


模型是基础

框架再好，底层模型不行，上限就在那里

就像你给一个**实习生**（差的模型）写了一份非常详尽的操作手册，他可能还是做不好。

但你给一个**高级工程师**（好的模型）同样的手册，他能做到远超你预期的水平。

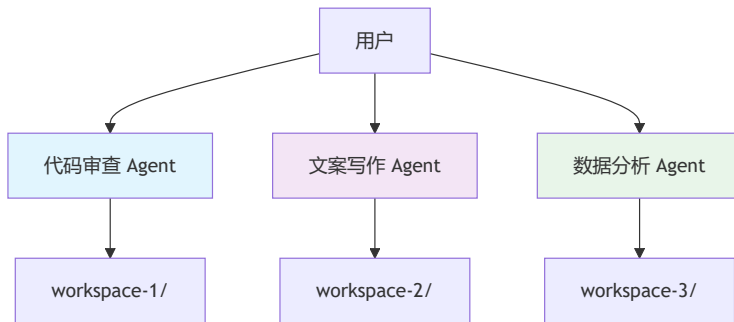


多 Agent 架构：专人专事

每个 Agent 有独立的：

- **workspace 目录** — 独立的知识空间
- **memory 数据库** — 独立的记忆存储
- **session 历史** — 独立的对话上下文

一个专做代码审查的 agent，积累的所有经验都是关于代码审查的，不会被写周报的对话污染掉。



01

OpenClaw 为什么越用越好用

Why OpenClaw Improves With Use

一个自我进化的 md 文件系统



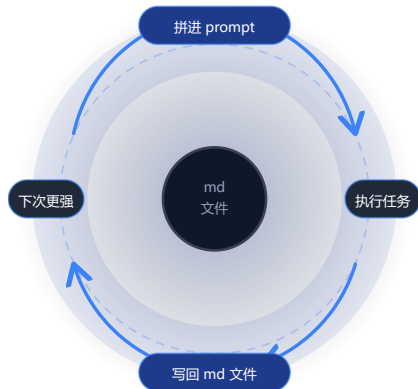
一句话概括

每次对话前，把一堆 md 文件**拼进**
prompt



对话后，让 agent 把新学到的东西**写**
回这些 md 文件

就这么简单。但这个简单的循环，构成了一个威力
巨大的飞轮。



骨架：7 个核心 md 文件

SOUL.md

Agent 是谁 — 人格、语气、风格、边界

USER.md

用户是谁 — 名字、时区、偏好、习惯

AGENTS.md

做事的规矩和踩过的坑

TOOLS.md

环境备忘 — SSH、设备、路径

SKILL.md × N

各领域的操作手册

memory/*.md

日常记忆（日记）

MEMORY.md

提炼后的长期记忆 — 从日记中整理出来的笔记精华

SOUL.md — Agent 是谁

定义 agent 的**人格**：语气、风格、边界、价值观。

源码模板里有一句很关键的话：

"This file is yours to evolve. As you learn who you are, update it."

也就是说，agent 的人格不是一次写死的，而是在长期互动中不断调整和沉淀。

SOUL.md文件负责什么

- 定义回答风格和表达语气
- 约束边界感与做事方式
- 逐步形成稳定的“角色感”

它决定的不是 agent 会不会做事，而是它会以什么样的方式和你协作。

USER.md — 用户是谁

这是 agent 对你的画像：名字、时区、工作习惯、技术偏好、沟通风格。

每次对话中只要它了解到关于你的新信息，就会更新这个文件。用得越久，画像越精准，agent 就越“懂你”。

USER.md文件通常沉淀

- 你偏好的输出形式
- 你常用的沟通语气
- 你的工作上下文和习惯

它解决的是“同样的能力，怎么更贴合这个用户”。

AGENTS.md — 最关键的文件

定义行为规范，更重要的是，**记录所有踩过的坑。**

源码模板里的明确指令：

- "When you learn a lesson → **update AGENTS.md**"
- "When you make a mistake → **document it** so future-you doesn't repeat it."

翻译成成人话：你犯了一个错，把它记下来，这样以后的你就不会再犯。

OpenClaw 越用越好用 —— 不是因为模型变聪明了

是因为 **AGENTS.md 里的踩坑记录越来越多**

每一条记录都是一次错误的代价换来的经验，被固化成了一行文字，从此永远生效。

TOOLS.md — 环境备忘

它记录你的工作环境：SSH 主机名、设备名、目录习惯、命令习惯，以及 agent 在真实执行中踩过环境坑。

这类信息往往很碎，但一旦缺失，agent 就会反复在同样的环境问题上浪费时间。

TOOLS.md文件的价值

- 让工具调用更少走弯路
- 让环境相关错误不再重复
- 把“本机经验”沉淀成长期上下文

很多看起来像“agent 不聪明”的问题，本质上只是环境信息没有被记住。

SKILL.md — 操作手册

内置 53 个 Skill (2026.4.9)

GitHub issue 管理、邮件处理、健康检查、代码审查...

更关键的是：你可以自己写

比如每周出一份特定格式的周报：

- 格式要求
- 数据来源
- 输出模板

写成 SKILL.md 放到 workspace → 从此 agent 每次都按规范来

一个关键变化

从“临时把 prompt 讲清楚”

变成“把做法长期写进 workspace”

所以 Skill 不只是插件集合，更像是可维护、可覆盖、可复制的操作手册系统。

SKILL.md — 优先级链

优先级链（低 → 高）

1. 插件提供的 skill
2. 内置 skill
3. 托管 skill (~/.openclaw/skills/)
4. 个人 skill (~/.agents/skills/)
5. 项目 skill (/agents/skills/)
6. **Workspace skill (/skills/)**

用户 workspace 里的 skill 优先级最高，可以覆盖任何内置行为。调教方式就是——写一个 md 文件。

这条链的实际意义

- 官方给你默认能力
- 个人和项目可以逐层覆盖
- 最终以当前 workspace 的规则为准

也就是说，最好用的 Agent 往往不是“默认最强”，而是“最贴合你当前工作空间”。

memory/*.md — 日常记忆

- 每天一个日期命名的 md 文件
- 记录当天对话要点、做了什么、学到什么
- 索引到 SQLite 数据库
- 支持全文搜索 + 向量检索

这类文件像“日记”

它记录的是原始事件流，而不是最后结论。

这些 daily memory 不会每次都完整塞进 prompt，而是先进入索引系统，等需要时再被检索出来。

MEMORY.md — 长期记忆

- 定期从 daily memory 中**提炼**重要内容
- 相当于从日记中整理出来的笔记精华
- **每次对话都会被加载进 prompt**
- Agent 的“长期记忆”就存在这里

它和 daily memory 的区别

- `memory/*.md` 更像原始素材
- `MEMORY.md` 更像提炼后的笔记
- 真正高频进入上下文的是这一页

这也是为什么 OpenClaw 能一边长期积累，一边控制 prompt 预算。

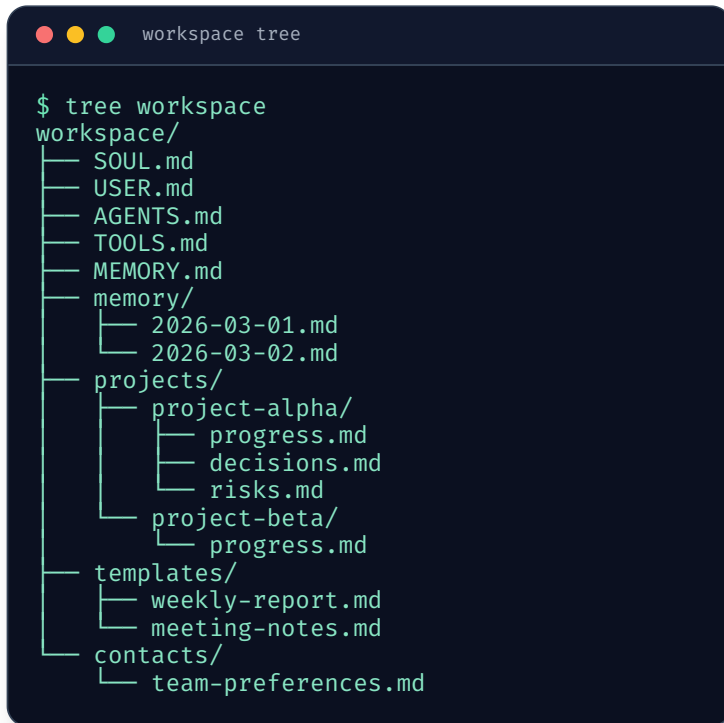
血肉：用户和 Agent 共同成长

7 类文件是框架预设的**骨架**。

但 workspace 本质就是一个普通文件夹，agent 有文件读写能力，可以创建**任何它需要的文件**。

每个人的 agent 最终长出来的形态不一样，取决于你跟它聊了什么、做了什么、在哪些领域用它。

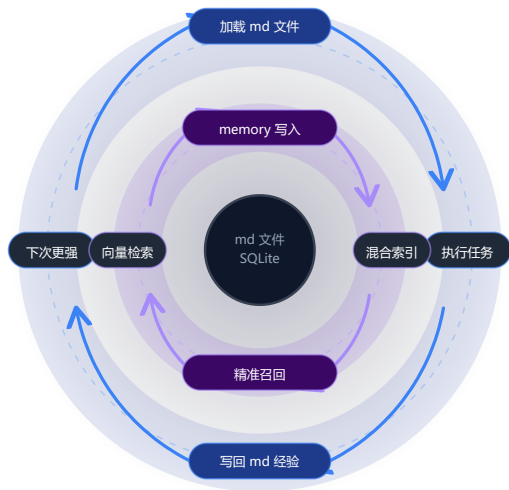
你的 agent 是真正“私人定制”的——不是勾选了几个选项，是通过几百次对话，自己生长出了一套只适合你的知识体系。



```
workspace tree

$ tree workspace
workspace/
├── SOUL.md
├── USER.md
├── AGENTS.md
├── TOOLS.md
├── MEMORY.md
├── memory/
│   ├── 2026-03-01.md
│   └── 2026-03-02.md
├── projects/
│   ├── project-alpha/
│   │   ├── progress.md
│   │   ├── decisions.md
│   │   └── risks.md
│   └── project-beta/
│       └── progress.md
├── templates/
│   ├── weekly-report.md
│   └── meeting-notes.md
└── contacts/
    └── team-preferences.md
```


进化闭环



外层循环 · md 文件读写

加载 md → 执行任务 → 写回经验 → 下次更强

经验层：知道该做什么、踩过哪些坑

内层循环 · 向量索引检索

memory 写入 → 混合索引 → 精准召回 → 再写入

检索层：FTS5 全文 30% + 向量检索 70%，按需召回

两层合在一起 = 完整的「学习 - 记忆 - 检索 - 应用」系统

技术细节：对话Bootstrap加载机制

`resolveBootstrapContextForRun()` 每次对话开始时：

1. 读取 workspace 下所有核心文件
2. 根据会话类型过滤（子 agent 只加载精简子集）
3. 允许插件通过 hook 修改内容
4. **每个文件限制 20KB，总量限制 150KB，超出部分截断**

预算机制的意义

md 文件不能无限膨胀 → agent 需要学会**"提炼"** → 把最重要的经验浓缩在有限空间里

这也是 MEMORY.md（提炼精华）和 memory/*.md（原始日记）要分开的原因。

技术细节：Memory 混合搜索

搜索权重

70%

向量相似度

30%

关键词匹配

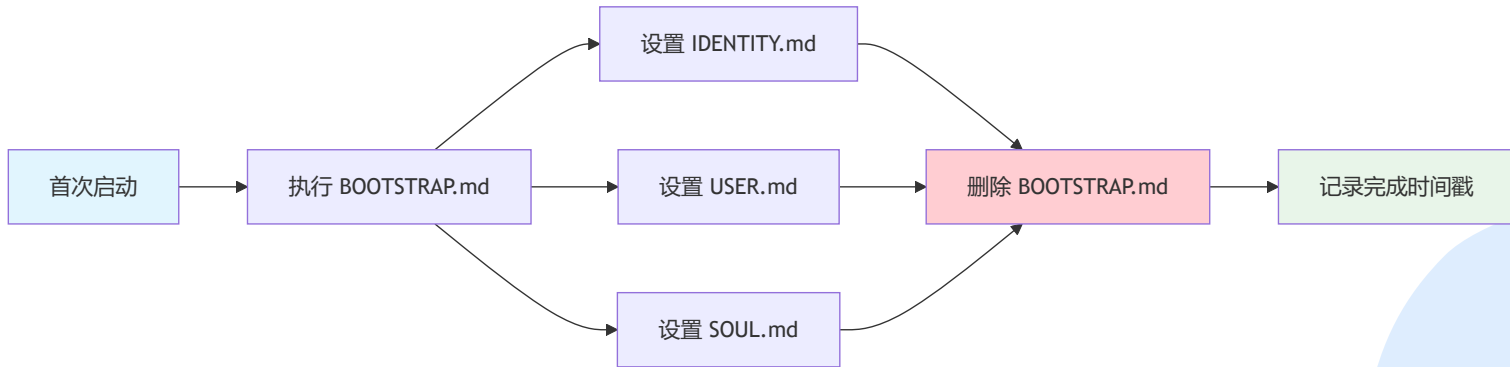
+

MMR 多样性 & 时间衰减

- **MMR** (Maximal Marginal Relevance) — 搜索结果尽量多样化，不会全是相似内容
- **时间衰减** — 最近的记忆权重更高
- 即使积累了几百个 memory 文件，也能精准找到相关信息

技术细节：首次自毁式引导

首次使用时的初始化流程：



一次性初始化，完成后自毁。Workspace 状态机记录引导完成的时间戳。

观点

1. 你的 Agent 的价值在 workspace 文件夹里

代码是公开的，模型是通用的。真正属于你的、不可替代的部分，是那堆 md 文件。换台电脑，拷过去就行；删掉，一切从零开始。

2. 调教 Agent 就是写 md

不需要学编程，不需要理解 prompt engineering。用自然语言写成 md 文件放到 workspace 里就行。甚至你不需要自己写，对话中 agent 自己会写，你只需要在它犯错时纠正它。

3. Agent 之间的差距就是 md 文件的差距

同样版本、同样模型，体验天差地别。差别在于 workspace 里积累了什么。跟现实世界一样，智商差不多，差距在于经验和知识。

4. 这可能是 AI Agent 产品的通用范式

"md 文件即知识"的架构有普适性。没有花哨的知识图谱，没有复杂的向量数据库集群，就是一堆 md 文件承载的不断进化的专家系统。

实操建议



主动引导 Agent 形成 SOP — 在你已有成熟工作流的领域，直接告诉它“以后这类任务都按照这个流程来”，让它写成 SKILL.md。



定期审查 workspace 文件 — Agent 自己写的东西不一定都对，定期看看 AGENTS.md、USER.md 里有没有过时或不准确的内容。



善用多 Agent — 不同领域配不同 agent，让知识积累保持垂直和纯粹。一个专做代码的 agent 比什么都做的 agent 好用得多。

实操建议



选对模型 — 一堆精心打磨的 md 文件喂给一个弱模型，效果也有限。模型是基础。



备份你的 workspace — 这是你最有价值的数字资产之一。建议用 Git 管理，定期推到远程仓库。

总结

OpenClaw 的源码有几十万行，但让它"越用越好用"的核心机制，说白了就是一个 **md 文件的读写循环**。

核心结论

代码决定了 OpenClaw 能做什么，md 文件决定了它做得多好。

代码提供了

渠道接入、模型调用、工具执行、memory 索引

md 文件承载了

你和 agent 一次对话一次对话攒出来的经验

02

OpenClaw 为什么出圈

Why OpenClaw Broke Out



OpenClaw 的关键动作之一

它把 Agent 带回了通信工具层

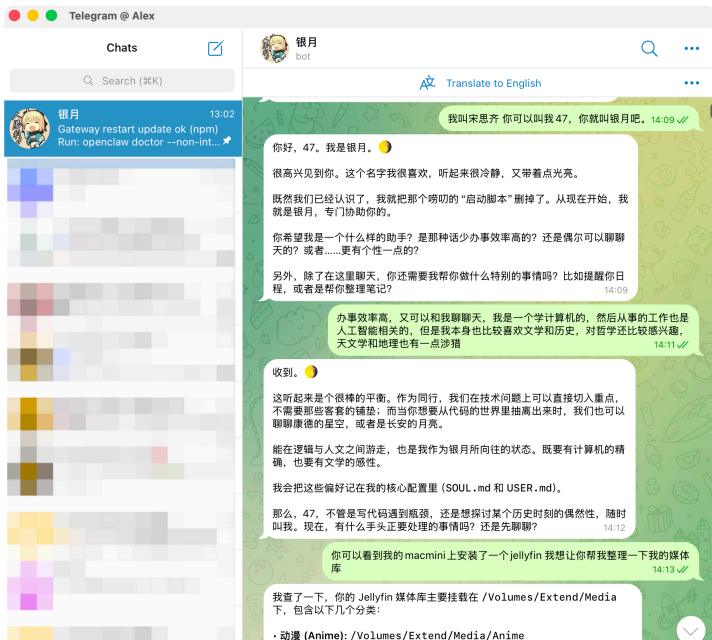
- 不再把 Agent 放在一个专门打开的工具里
- 而是放回用户本来就在用的消息入口
- 会话、工具、设备和扩展能力被放到了一起

“对很多用户来说，这不是「多了一个 AI App」，而是「多了一个常驻同事」。”

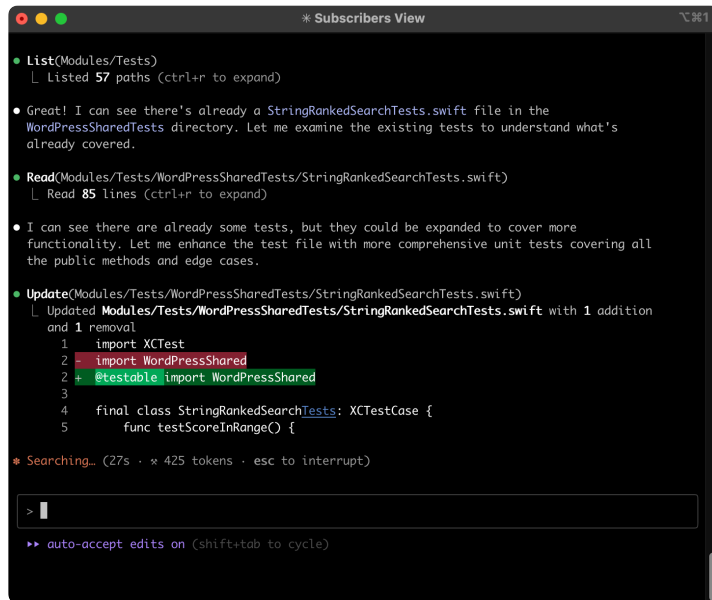


它不是另一个开发者工具

OpenClaw



Claude Code



OpenClaw 出圈，靠的三个产品杠杆

1. **Skill** 把能力做成可复用模块，不再只靠 prompt
2. **IM 入口** 让调用动作发生在高频沟通场景里
3. **常驻在线** 让用户开始把它当成一个持续协作对象

这三件事叠在一起，才让 Agent 从"能演示"走向"想天天用"。

Skill 机制，是 OpenClaw 的第一块 地基

经验终于可以被打包、复用和扩展

my-skill/

└─ SKILL.md

核心指令：触发条件、任务流程、执行指引

└─ scripts/

可执行代码：AI 直接运行的固定脚本程序

└─ references/

参考文档，给 AI 看的：技术规范、API 文档、专业指南

└─ assets/

素材资源，拿来用的，会被复制、修改：模板、图片...

- 一段好 prompt 只能临时奏效
- 一个 Skill 才能把说明、工具、脚本和资料绑在一起
- 这意味着经验开始能沉淀，不必每次重讲一遍

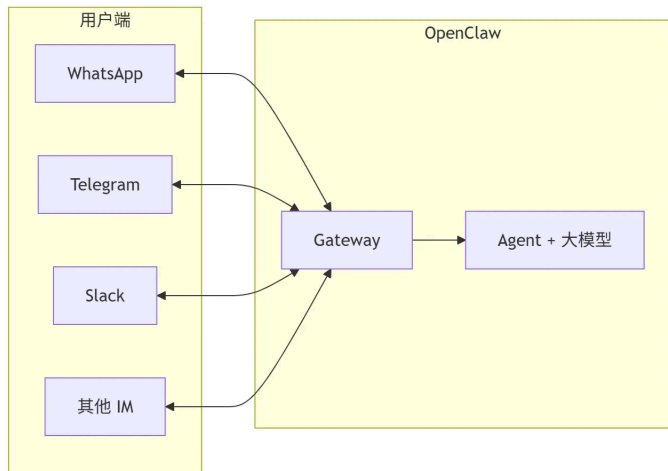
“对开发者来说，Skill 的价值不只是方便，而是把“会做”变成“可复制”。”

消息入口，决定了使用频率

Agent 越接近日常沟通入口，越容易进入真实工作

- 用户不用切到另一个页面再重新描述需求
- 在私聊、群聊和协作 IM 里叫它做事，门槛更低
- 一旦形成习惯，Agent 就不再是低频工具

“很多产品差异，最后都体现在“你愿不愿意每天都打开它”。”

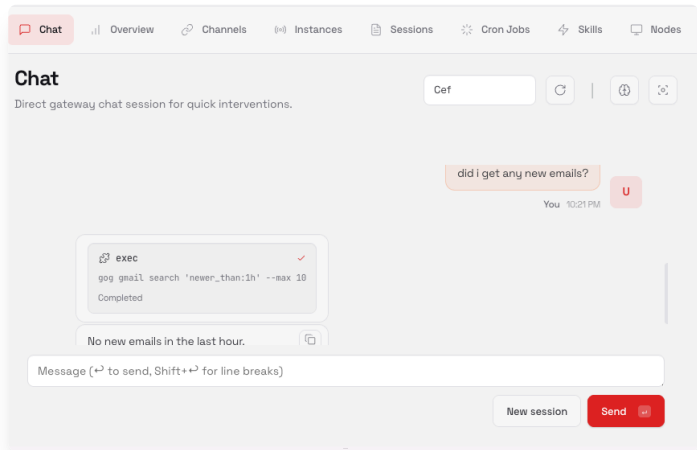


常驻在线之后，用户会按"同事标准"要求它

不只是问答，而是持续协作

- 能读文件、跑命令、调浏览器、接 API
- 能被定时触发，也能被事件唤起
- 能在任务没做完时继续往下推

“这时大家期待的就不再是“回复我”，而是“把这件事继续推进”。 ”

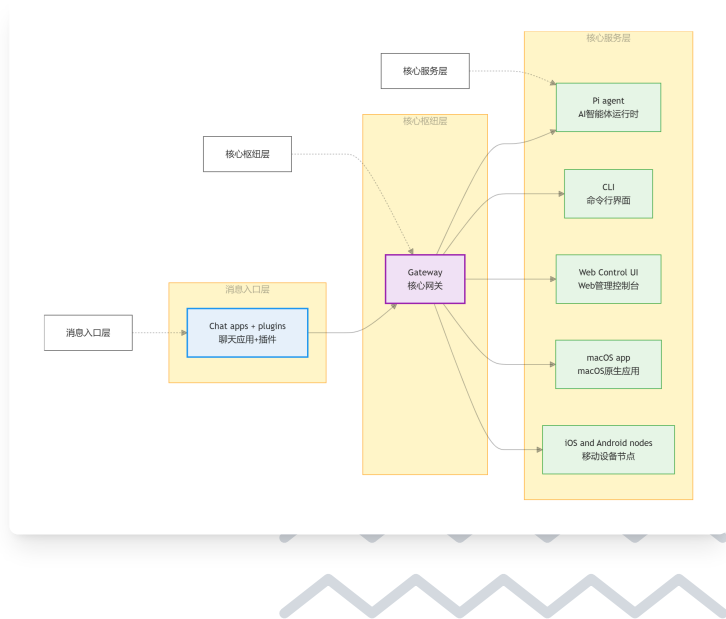


所以 OpenClaw 本质上是一套控制平面

真正值钱的，不是一项单点能力

- 消息入口、Gateway、Agent、Web UI、设备节点被接成了系统
- 任务路由、工具调用、会话状态开始在同一条链路里流动
- 这也是它能从聊天入口走向执行入口的基础

“看懂这张图，就能明白它为什么不像一个插件，而像一套平台。”

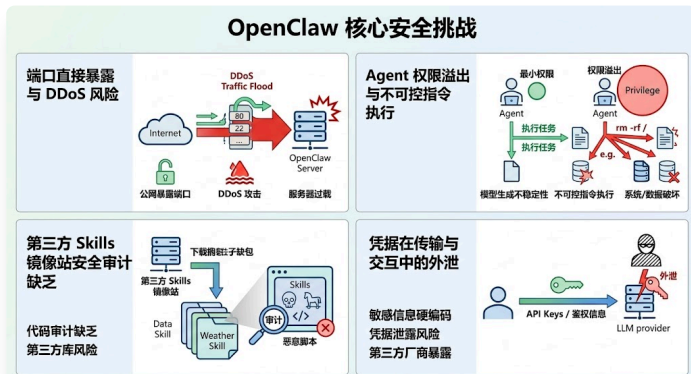


但一进真实环境，安全和治理马上跟着来

Agent 一旦能调用系统能力，风险就不再是边角问题

- 权限边界怎么控
- 敏感信息怎么隔离
- 恶意 Skill、错误执行和暴露面怎么收口

“这也是为什么“能跑起来”和“能大规模用起来”，中间还隔着一层平台能力。”



03

AstronClaw 为什么更适合长期使用与落地

Why AstronClaw Is Better for Long-Term Use

AstronClaw 要解决的

不是“再多一点自由”
而是“谁来接住复杂度”

OpenClaw 更像高自由度底座，AstronClaw 更像把部署、入口、治理和长期运维一起接住的平台产品。

平台托管，先把部署和运维收走

这是从极客玩法走向普遍可用的第一步

- 网页可启用，门槛更低
- 平台 7×24 在线，用户不用自己盯环境
- 隔离执行、统一治理，比个人自部署更容易进标准流程

“对很多团队来说，真正稀缺的不是模型，而是能不能少折腾。”



本地化 IM 接入

入口不是装饰，而是能不能进入真实工作的关键

- 飞书、钉钉、企业微信、微信等入口更贴近日常沟通
- 这决定了个人、小团队和业务团队能不能顺手接住它
- 对国内场景来说，入口适配本身就是产品力

“一款 Agent 产品有没有机会留下用户，要看它能不能进入现有沟通习惯。”

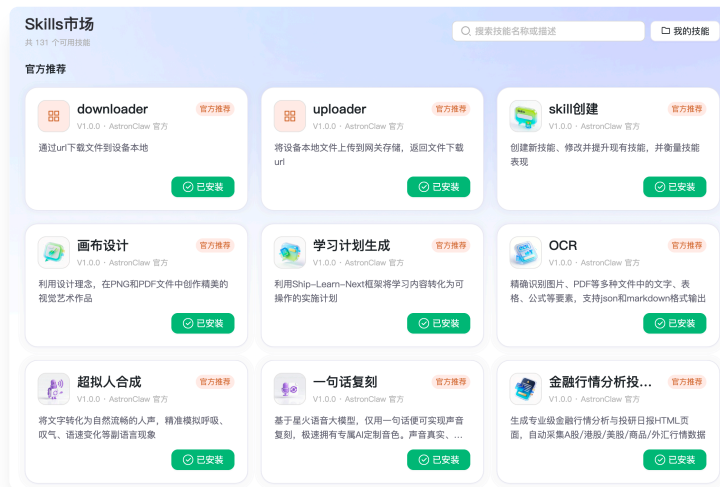


Skill 数量只是开始

从“能调用能力”到“能治理能力”

- 官方市场降低开箱门槛
- 上传文件即可创建 Skill，便于沉淀个人和团队经验
- 能力开始可复用、可组合，也更容易做审计和协作

“到这一步，Skill 才不只是插件，而开始像组织资产。”

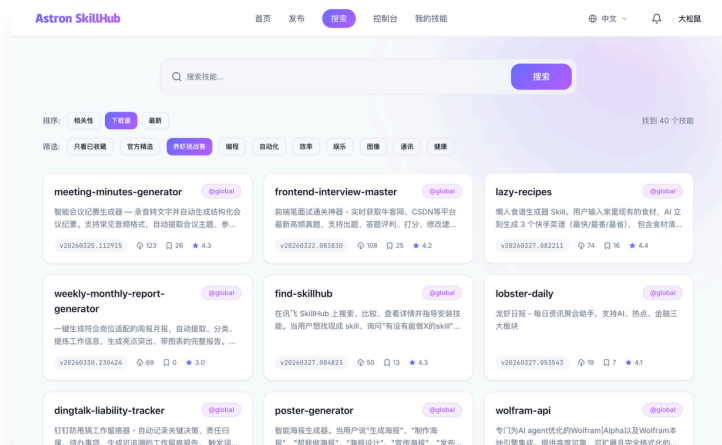


Astron Skillhub 让 Skills 变成长期资产

这可能是企业最看重的，也是个人用户最容易忽视的

- Skill 可以发布、搜索、版本化、复用
- 团队经验不再散落在个人环境里
- 权限、命名空间和协作机制等特性

“单个 Skill 解决一个任务，SkillHub 解决的是能力怎么长期沉淀。”

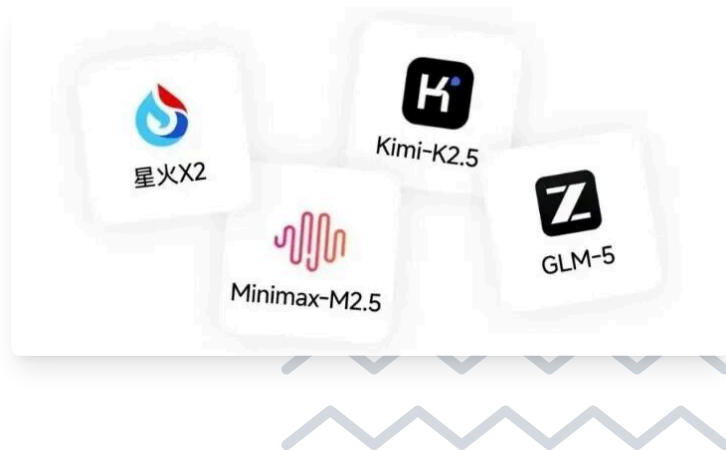


多模型可选

让不同任务可以用不同底座

- 有的任务更看重推理
- 有的任务更看重成本
- 有的任务更看重安全和可控

“真正有价值的不是模型越多越好，而是任务匹配更合理。”



它已经进入真实沟通入口

星火派



适合定时任务和固定信息分发

Astron 小助手



适合问题解答和一线服务支撑

这说明它不只是演示形态，而是已经进入真实沟通链路。

感谢聆听



扫码加入交流群

相关链接:

- OpenClaw: <https://github.com/openclaw/openclaw>
- AstronClaw: agent.xfyun.cn
- SkillHub: github.com/iflytek/skillhub